

Explicación a Profundidad: Recursividad en Python

Análisis del algoritmo de suma de dígitos

Este fragmento es un ejemplo clásico, elegante y muy eficiente de recursividad en Python. Tienes toda la razón al intuir que `% 10` saca el último dígito, pero vamos a desarmar exactamente por qué sucede eso matemáticamente y cómo funciona la función paso a paso.

La Matemática detrás del Código

Para entender esta función, primero debemos mirar los dos operadores matemáticos que hacen todo el trabajo pesado: el módulo (%) y la división entera (//). Como trabajamos en un sistema de base 10 (el sistema decimal), el número 10 es la clave.

1. El operador Módulo: `n % 10`

El operador % calcula el residuo o resto de una división.

Matemáticamente, si divides cualquier número entre 10, el sobrante de esa división exacta siempre será el último dígito de tu número original.

Por ejemplo, tomemos tu número, 142:

$$142 = (14 \times 10) + 2$$

Como el 2 es lo que sobra después de hacer los grupos de 10, el cálculo en Python `142 % 10` te devuelve exactamente 2. Esta es la forma matemática de "arrancar" el dígito de la derecha.

2. La división entera: `n // 10`

El operador // realiza una división pero descarta los decimales, quedándose solo con la parte entera (lo que se conoce como truncar hacia abajo).

Matemáticamente:

$$\frac{142}{10} = 14.2$$

Al usar la división entera `142 // 10`, Python ignora el .2 y te devuelve 14. En la práctica, esto es el equivalente a "borrar" el último dígito del número que acabas de extraer en el paso anterior, dejándote el resto del número listo para ser procesado.

Cómo Funciona la Recursividad

La recursividad ocurre cuando una función se llama a sí misma para resolver una versión más pequeña del mismo problema. Toda función recursiva necesita dos cosas para no quedarse atrapada en un ciclo infinito:

- **El Caso Base** (`if n == 0: return 0`): Es el freno de emergencia. Le dice a la función: "Si ya no quedan más dígitos por sumar (es decir, el número se ha reducido a 0), detente y devuelve 0".

- **El Paso Recursivo** (`return (n % 10) + sumar_digitos(n // 10)`): Aquí es donde ocurre la magia. La función dice: *"Voy a tomar el último dígito de este número, y le voy a sumar el resultado de mandar el resto del número a esta misma función"*.

Prueba de Escritorio (Dry Run) con 142

La recursividad tiene dos fases: una **fase de ida** (donde las funciones se abren y quedan en pausa esperando respuestas) y una **fase de vuelta** (donde se entregan los resultados y se resuelven las sumas).

Fase de Ida (Llamando a la función)

Arrancamos ejecutando `sumar_digitos(142)`:

1. **Primera llamada:** $n = 142$

- ¿Es 142 igual a 0? No.
- Ejecuta: $142 \% 10$ (que es 2) + llama a `sumar_digitos(14)`
- El 2 se queda "en pausa" esperando a que la otra función termine.

2. **Segunda llamada:** $n = 14$

- ¿Es 14 igual a 0? No.
- Ejecuta: $14 \% 10$ (que es 4) + llama a `sumar_digitos(1)`
- El 4 se queda en pausa.

3. **Tercera llamada:** $n = 1$

- ¿Es 1 igual a 0? No.
- Ejecuta: $1 \% 10$ (que es 1) + llama a `sumar_digitos(0)`
- El 1 se queda en pausa.

4. **Cuarta llamada:** $n = 0$

- ¿Es 0 igual a 0? ¡Sí! Hemos llegado al caso base.
- La función retorna **0** y no se llama a sí misma de nuevo.

Fase de Vuelta (Resolviendo las pausas)

Ahora, ese 0 que nos devolvió el caso base viaja de regreso hacia arriba, completando las sumas que habíamos dejado en pausa:

- La **tercera llamada** estaba esperando: $1 + \text{sumar_digitos}(0)$. Como ahora sabe que eso vale 0, resuelve $1 + 0 = 1$. Retorna 1.
- La **segunda llamada** estaba esperando: $4 + \text{sumar_digitos}(1)$. Como ahora sabe que eso vale 1, resuelve $4 + 1 = 5$. Retorna 5.
- La **primera llamada** estaba esperando: $2 + \text{sumar_digitos}(14)$. Como ahora sabe que eso vale 5, resuelve $2 + 5 = 7$. Retorna 7.

El resultado final que ves en tu pantalla es **7**, que es exactamente la suma de $1 + 4 + 2$.